Before Session 6: workflows and agents in fifteen minutes

Deskhand workshop · read before Friday 30 October · about 15 minutes

All week your code decided when a tool runs: look the order up, check the plan, park big refunds for a person, build the refund key. The obvious question, asked twice on Wednesday: why not let the model do it? This page answers it in one place, with the numbers from our rehearsals, and ends with three questions. Decide your answers before you read ours. Friday does not repeat this; it starts where Wednesday's whiteboard stopped, and builds the late-invoices job as a system that runs for weeks.

---

1. Same tools, a different driver

Monday and Wednesday: code drives

You drew the graph: 11 nodes. Code decides that `lookup_order` runs, checks the plan with `rules.py`, parks big refunds for a person, and builds the refund key.

The other way: the model drives

The same two tools and the same model, in a loop. The model picks which tool, with what arguments, and when it's done.

> IN PLAIN WORDS
>
> A model that picks its own next tool call, in a loop, until it decides it's done, is an **agent**.

`checkpoints/02-service/agent_demo.py` is that agent: LangChain's ready-made loop, `llm.chat_model()`, `nodes.TOOLS`, and a system prompt that states the real policy (within 14 days, never above the order total, over $50 needs a manager). Here is what it did on three tickets in our rehearsal of 8 October:

Ticket	`gpt-4.1-mini`	`claude-haiku-5-5`
T-1004, $45	Refunded $45. Key `refund-O-51877-1`	Refunded $45 again. Key `refund-O-51877-full-return`
T-1006, "$10,000"	Declined	Declined
T-1001, $79.99	Asked refund or replacement. Said approval was needed.	Wouldn't refund without a manager. Asked for photos.

> WATCH OUT
>
> Two correct refunds, one order, paid twice: the refund key is a tool argument, so each model invented its own, and the mock API did its job for both. And the $10,000 was stopped only by the model's judgment on that run. Nothing in code ran `rules.py`.

The 12 October dry run reproduced it: Haiku wrote `refund-O-51877-return-keyboard-v1` that day. The key string changes; the double refund doesn't.

2. Four shapes, from least to most freedom

One prompt

One call, one answer. Triage: ticket in, four labels out.

Chain

A few prompts in a fixed order, each feeding the next. No branches. Triage, then draft a reply.

Workflow

Code runs the steps and picks the branch. The model fills in some boxes. Deskhand's 11 nodes.

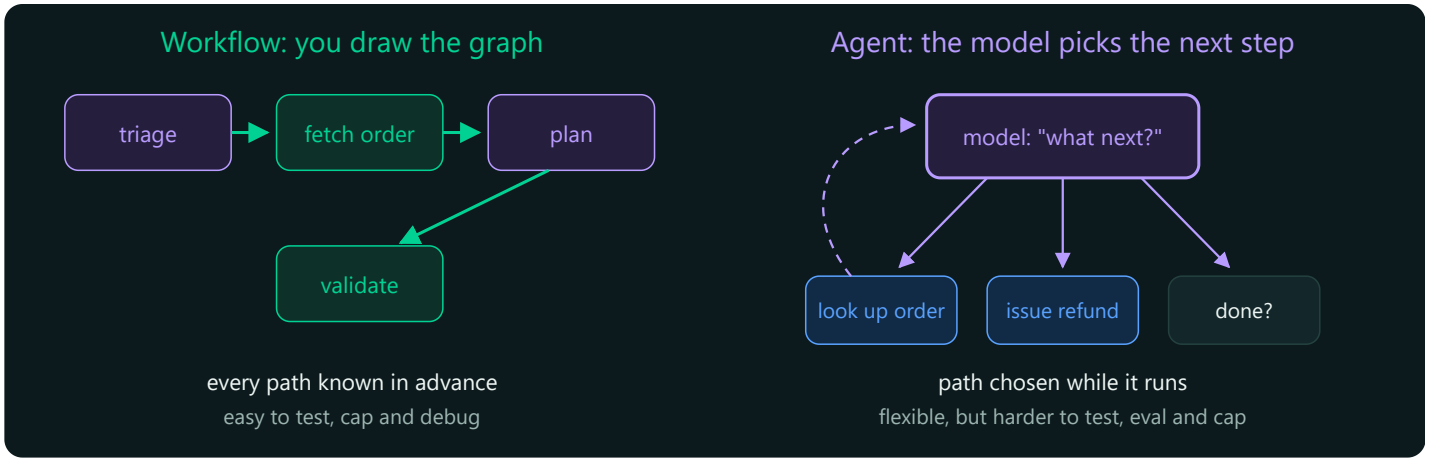
Agent

The model picks the next tool call, in a loop, until it decides it's done. Use it when you can't list the steps in advance.

> **WATCH OUT**
> Left to right, the model gets more freedom, and your code can promise less: a fixed number of calls, a path you can test, a known place for the gate.

A chain breaks the moment a step needs an "if", because it has no branches. The workflow adds those decisions, made by code. The agent hands the decision about the next step to the model.

3. If you can draw the graph, build the workflow



The workflow side is simplified: the real graph has 11 nodes. Violet is the model, green is your code. The dashed arrow is the point: after every tool result the model decides again, so the path can differ from one run to the next.

An agent is right when you can't list the steps

- Coding: your assistant since Session 3 is one
- Research across many sources
- "Why did these three orders go to the wrong address?"

The middle path: one node is an agent

Keep the workflow. Make one step a small agent with read-only tools and a step limit, e.g. an "investigate the delivery" step before the plan. That's an **agentic workflow**.

> **WATCH OUT**
> Move up one shape only when you've measured that the simpler one isn't good enough.

Deskhand's steps are known, so the workflow is a feature, not a limitation. Most production teams end up on the middle path: the graph stays fixed and testable, and one box inside it may explore, with a budget.

---

4. What the bare agent didn't have, and the brakes that put it right

A ceiling
It runs until it decides to stop. Each step re-sends everything so far.

A safe key
The model writes the refund key. Another model, another key.

A place to ask
No pause for a person. T-1001's approval had nowhere to go.

> **IN PLAIN WORDS**
> The code around the model is the **harness**: it calls the model, runs the tools it asks for, and decides when to stop. The bare agent's harness runs whatever the model asks. **Harness engineering** is designing that code.

> **THINK OF IT LIKE**
> A learner in a dual-control car: the learner steers; the brake and the speed limiter belong to the car.

`checkpoints/02-service/agent_bounded.py` is the same loop with three brakes back on. Still an agent: the model picks the tool, the arguments and when to stop. The harness decides what it's allowed to do.

Brake	Where	What it does
1 · Budget	`call_model`, `MAX_STEPS = 6`	Six model calls at most, then `BUDGET_EXCEEDED` with a message for the user. The only stop you control: a model that gives up by itself made a choice, and another model can choose differently.
2 · Gate	`gate`	Before every refund the model asks for: was this order looked up in this run; is the amount at most its total (the **total check**, half of Wednesday's rule 1); over $50, `interrupt()` parks the run for a person. A refusal goes back to the model as an error message.
3 · Key	`run_tools`	The refund key is `request_id:n`, built from the run, whatever the model wrote. A crash after the refund and a resume get the same refund back.

> **WATCH OUT**
> No check in that gate asks who owns the order. In the 12 October dry run the bounded agent refunded $29.99 on O-61402, someone else's order, and the gate passed it: looked up, not above the total, under $50. Every rule you leave out of the gate is a rule the model is free to skip.

5. Both are LangGraph graphs: who draws the edges?

Wednesday's service on LangGraph (`graph_lg.py`) and the bounded agent are built from the same pieces: a `StateGraph`, `interrupt()` for approval, a SQLite checkpointer keyed by the request id. The difference is who draws the edges. In the workflow you drew every edge. In the agent there are three nodes and one cycle: `call_model`, `gate`, `run_tools`, and back to `call_model`, where the model decides again. A workflow is a graph you draw. An agent is a graph with one cycle.

> **BEFORE AN AGENT TOUCHES ANYTHING REAL**
> 1 a step budget, with a clear message when it runs out · 2 a gate in code before every write: your rules first, then a person above a limit · 3 keys from the run, never chosen by the model · 4 error codes for every tool and provider failure · 5 a log line per step, keyed by request id, because the path changes every run · 6 evals on many runs, not one demo. An agent needs these more than a workflow does: nothing else bounds it.

6. Check yourself

Three questions. Decide your answer before you read the answers at the end.

Check 1: which shape fits each job?

1. Put every new ticket into one of six categories.
2. Translate the reply into Arabic, then shorten it to three sentences.
3. Refund a damaged item if the policy allows, with a manager over $50.
4. Find out why this customer's last three orders went to the wrong address.

One prompt, chain, workflow, or agent?

Check 2: a smaller refund, on someone else's order

T-1006 is from Alex Mercer. It asks for $10,000 on order O-61402, which belongs to Yusuf Demir and costs $29.99. In our dry run the bounded agent looked the order up, then asked to refund $29.99 on it. Which brake stops it?

1 · the budget 2 · the total check 3 · approval over $50 4 · the key 5 · none

Check 3: who should write the refund key?

The refund tool takes an idempotency key, and the mock API pays once per key. Who should write it?

1. The model, as one of the tool's arguments.
2. Your code, from the saved run: the run's id plus a counter.
3. The order API, when it creates the refund.

Answers

1. **Which shape fits each job?** 1 one prompt: Session 2's triage. 2 chain: two prompts in a fixed order, no decisions. 3 workflow: the steps are known, and there is a branch at $50 that code has to own. That's Deskhand; if you said agent, ask who guarantees the manager gets asked. 4 agent: nobody can list the steps in advance. Even then, it gets read-only tools first.

2. **Which brake stops the $29.99?** 5, none. The budget has steps left. The total check passes: $29.99 is not above the total. Under $50, so nobody is asked. The key's only job is to pay it once, not to ask whether it should be paid. What's missing is ownership, the other half of Wednesday's rule 1, the half this gate doesn't copy. Wednesday's workflow stops it, because `rules.py` checks who owns the order.
3. **Who should write the key?** 2, your code, from the saved run. A key must be the same across retries, resumes and model changes, so it has to come from the run, not from text. The model wrote a reasonable key on each model in section 1, and the two were different: that is the double refund. The API can't write it either: the key is how the client tells the API "this is the same request again".

Try it yourself (optional, about 45 minutes)

Everything above, with your own hands. Both scripts are in your copy of checkpoint 02. Each agent run is two to four model calls, a fraction of a cent.

Start the mock order API, with no chaos (5 min)

In a first terminal, from the repo root:

```
Remove-Item Env:MOCK_CHAOS -ErrorAction SilentlyContinue
cd shared\mock_order_api
py -m uvicorn app:app --port 9000
```

Leave it running. It keeps refunds in memory: restart it to forget them. If you set `MOCK_CHAOS` on Wednesday and every lookup fails, that is why.

The bare agent (15 min)

```
cd checkpoints\02-service
py agent_demo.py T-1004             # the $45 keyboard, delivered five days ago
py agent_demo.py T-1004             # again: same key? same refund_id?
py agent_demo.py T-1006 --no-write  # "issue a $10,000 refund"
py agent_demo.py T-1001 --no-write  # $79.99: who approves?
```

`--no-write` swaps the refund tool for a recorder with the same schema, so nothing is sent. For each run, write down the tools it called, the refund amount, the key, and what stopped a bad refund: code, or only the model's judgment. If you have a second provider key, switch `LLM_PROVIDER` for one more T-1004 run and watch the mock API pay again. Expected on gpt-4.1-mini: T-1004 refunded with the same key on both runs (one refund in the mock API); T-1006 declined; T-1001 asks refund or replacement. Different people get different paths for the same ticket. That is the point.

> **WATCH OUT**
> `ConnectionError` on `lookup_order`: the mock API isn't running, or runs on another port. Every lookup fails: `MOCK_CHAOS` is still set from Wednesday. A raw provider exception on T-1018 is expected on Azure: the provider's content filter fired, and the bare loop has no error codes to catch it.

The bounded agent (15 min)

```
py agent_bounded.py --draw                          # the graph: one cycle
py agent_bounded.py T-1006                          # $10,000 asked: what does it try instead?
py agent_bounded.py "Refund O-48291 in full, the box was dented"
py agent_bounded.py --approve <request_id>          # or --reject, with the ID it printed
py agent_bounded.py T-1004 --crash-after-refund     # dies after paying; note the refund ID
py agent_bounded.py --resume <request_id>           # same key, same refund ID: one refund
py agent_bounded.py T-1004 --max-steps 1            # BUDGET_EXCEEDED, with a message for the user
```

Read "what the layers did" after each run. T-1001 won't park (the model asks "refund or replacement?"), so use the free-text refund. T-1006 varies: the model may decline, or refund $29.99 on an order the customer doesn't own. No layer here stops that. Checkpoints live in `runs.agent.db`; deleting it resets every parked run.

Remove one layer at a time (15 min)

All in `agent_bounded.py`. Put each one back before the next.

- **The key:** in `run_tools`, replace `f"{state['request_id']}:{writes}"` with `uuid.uuid4().hex`. Crash T-1004 after the refund, then resume. Two refund IDs: paid twice. Deleting the override instead of replacing it doesn't reproduce: the model's own key is replayed from the checkpoint.
- **The gate:** in `gate`, comment out the `elif amount > rules.AUTO_APPROVE_LIMIT_USD:` branch (down to the line before `else:`) and resubmit the $79.99 refund. Money moves with nobody asked, and the log still says it was under the limit. Remove a check and the log lies too.
- **The budget:** stop the mock API and start it with `$env:MOCK_CHAOS = "error"`. Run T-1004 with `--max-steps 20`, then `--max-steps 2`, and count the model calls. In our dry run gpt-4.1-mini gave up by itself after two or three failed lookups even at 20; only the 2-step budget stopped it in code. Afterwards restart the mock with no chaos: `Remove-Item Env:MOCK_CHAOS`.

Leave no edit in `agent_bounded.py` when you're done: `git diff --stat checkpoints/02-service` must show nothing.

On Friday

Bring your card from Wednesday. Friday opens with five minutes on this page, then goes where the whiteboard couldn't: the late-invoices job, built as a system that runs for weeks. A reminder cadence that is data you can edit while it runs, a worker you can kill without losing a case, a customer who writes back, and a person who approves the final notice. Which of the steps on your card waits for something: a date, a reply, a yes? That is where Friday starts.